

CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 23 Part 2: Operating Systems

Instructor: Justin Yokota

Today's Agenda

- Operating Systems and IO
 - Managing Multiprocessing: Interrupts, Exceptions, Trap Handler
 - Input/Output
 - Boot

Today's Agenda

- **Operating Systems and IO**
 - Managing Multiprocessing: Interrupts, Exceptions, Trap Handler
 - Input/Output
 - Boot

Overview

- So far, all our discussion has been from the perspective of one program.
- From the program's perspective, the following are true:
 - "I am the only program running on the CPU"
 - "There's only one CPU, one memory, etc. on this system"
 - "I have a full 2^{32} bytes of memory to use however I want"
 - For 64-bit systems, memory is 2^{48} bytes
 - "ecalls (ex. printf, malloc, scanf) just work"
 - "All my data's inputs/outputs are in the form of strings, and I get them/output them through stdout/in (or files)."
- This isn't actually true anymore
 - Citation: My computer is currently running a bunch of different processes (well more than the 12 CPUs I have), none of which are just a text-based interface, and none of which are using anywhere close to a TiB of memory.

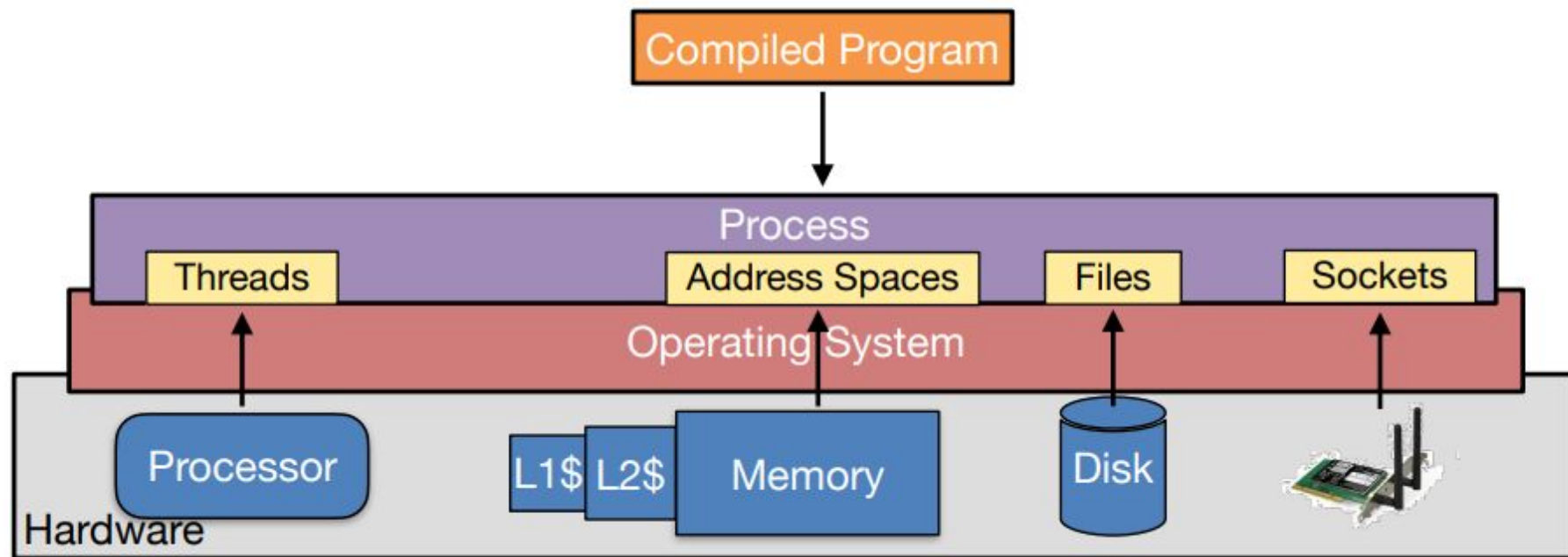
Computer Startup

- Solution: set up one process whose main purpose is to coordinate other processes, and run it on startup. This is known as the Operating System, or OS
- How this works:
 - When you start the computer, the system loads a startup program known as the BIOS (Basic Input/Output System), which is often stored on a special read-only chip
 - The BIOS does some quick checks to make sure the computer can run, then loads the OS from memory, then transfers control to the OS.
 - The OS is then responsible for running other programs.
- Example OSes: macOS, Windows, Ubuntu, BSD

The OS

- The Operating System acts as:
- An illusionist
 - Any program we run doesn't need to know that the OS or other programs exists
 - Any program we run doesn't need to worry about how syscalls actually work

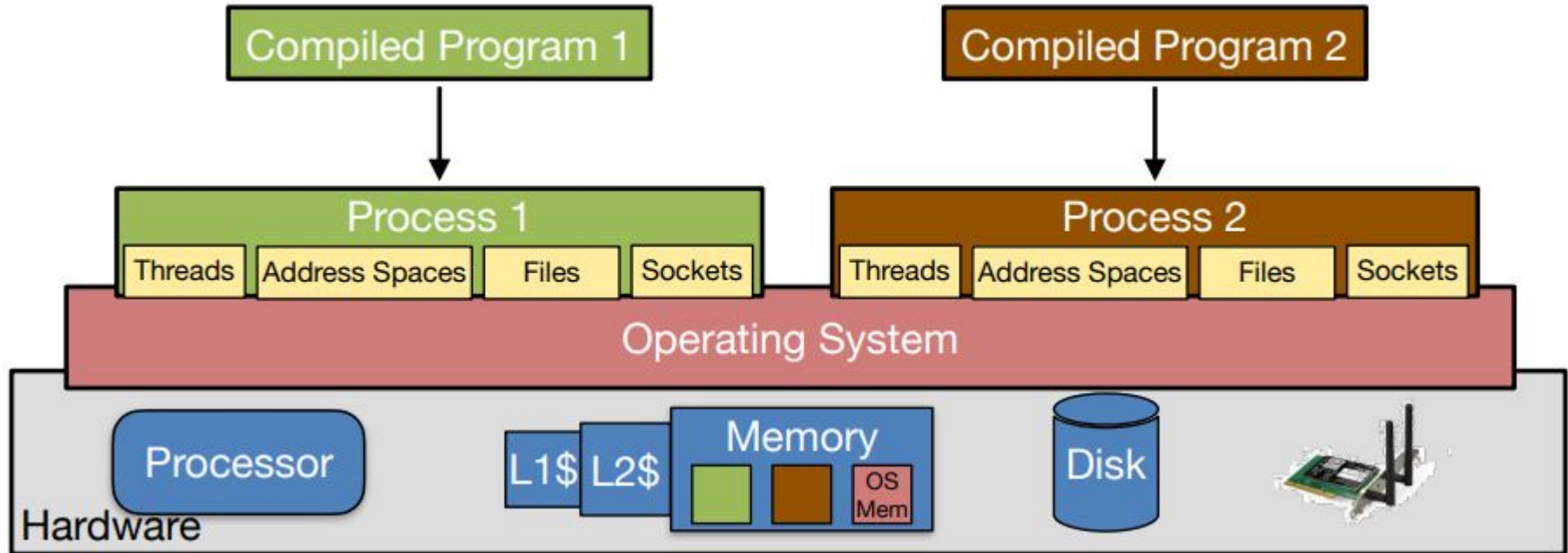
The OS



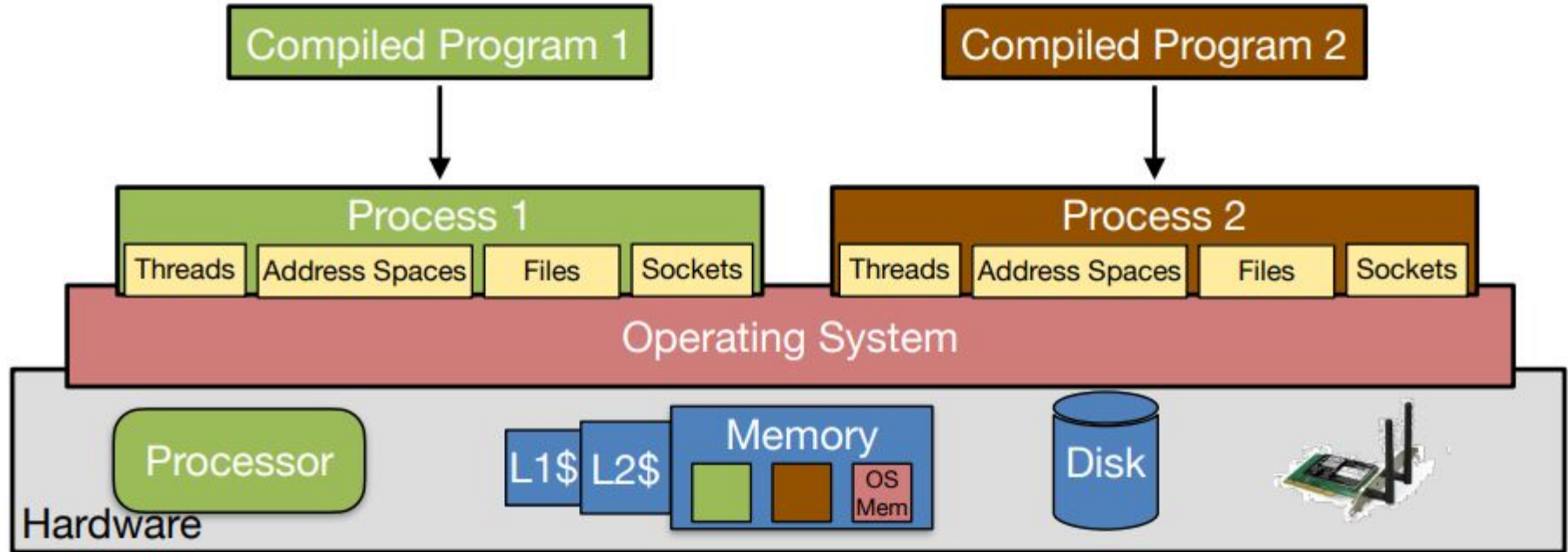
The OS

- The Operating System acts as:
- An illusionist
 - Any program we run doesn't need to know that the OS or other programs exists
 - Any program we run doesn't need to worry about how syscalls actually work
- **A conductor**
 - Receive commands from the user and assigns computer resources to tasks

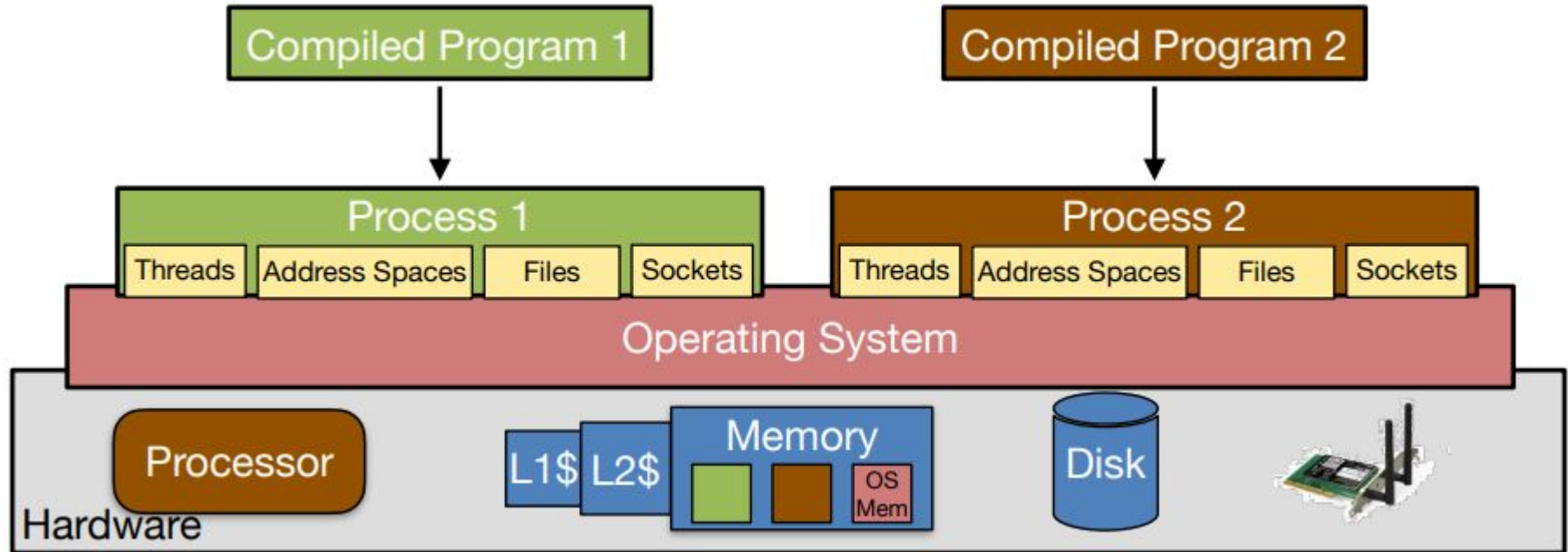
The OS



The OS



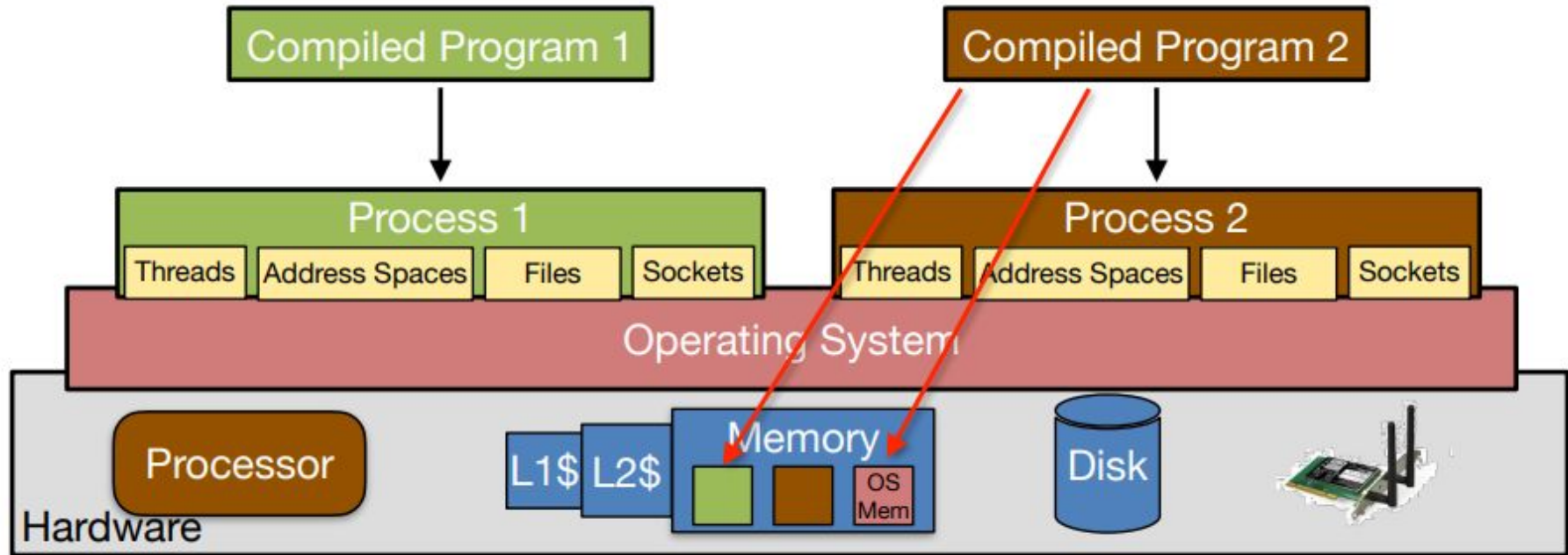
The OS



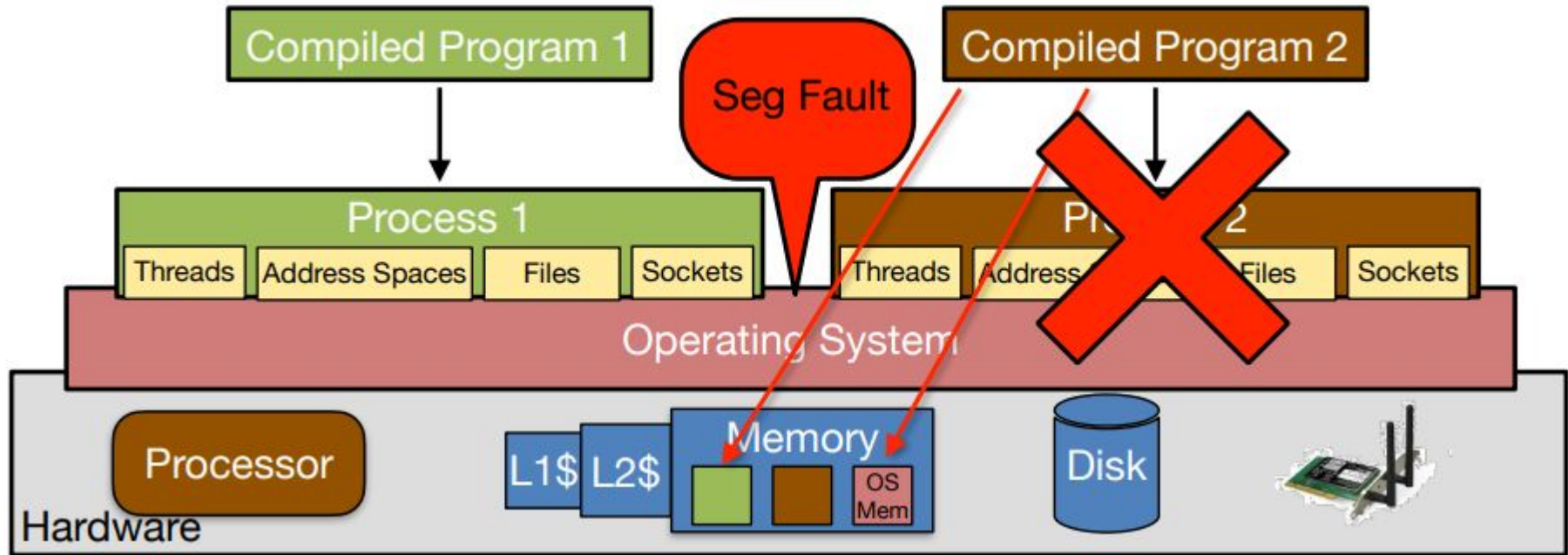
The OS

- The Operating System acts as:
- An illusionist
 - Any program we run doesn't need to know that the OS or other programs exists
 - Any program we run doesn't need to worry about how syscalls actually work
- A conductor
 - Receive commands from the user and assigns computer resources to tasks
- A referee
 - Keep track of what processes are running, and assign appropriate permissions
 - If a process exceeds what it's allowed to do or crashes, terminate the process without crashing the entire system.
 - This includes preventing different programs from affecting each other, and preventing programs from affecting the OS itself.
 - If a program's using a lot of one resource, make sure other programs also get a turn

The OS



The OS



Dual Mode Operation

- Generally, the system provides at least two different "permission levels".
 - "User Mode", which has limitations on what can be accessed, but is generally "safer" to use
 - "Kernel Mode", which has more freedom, but which can crash the entire system if it fails
- Goal: Restrict "Kernel Mode" to the OS so the user doesn't get to do whatever it wants (helps protect from malicious code)
- Goal: Restrict "Kernel Mode" to as few parts of the OS as possible, so if a part of the OS gets compromised, bad things don't happen as easily.
- Shell scripts (ex. Git Bash) allow us to communicate directly with the OS (instead of through a program), thus bypassing some of the protections.

Dual Mode Operation: User, Kernel

- Hardware **supervisor mode** (i.e., **kernel mode**) enforces constraints to other processes.
 - e.g., access to memory, devices, etc.
 - Set by a status bit in a special register.
 - Errors in supervisory mode are often catastrophic.
 - blue "screen of death"
 - "I just corrupted your disk"
 - In HW—not "superuser"!

- **User mode**: A process can only access a subset of instructions and (physical) memory.
 - Can change out of supervisor mode using a special instruction (e.g., `sret`).
 - Cannot change into supervisor mode directly; instead, HW interrupt/exception.

The OS mostly runs in user mode!
Supervisor mode is used sparingly.

Today's Agenda

- **Operating Systems and IO**
 - **Managing Multiprocessing: Sharing Resources**
 - Interrupts, Exceptions, Trap Handler
 - Input/Output
 - Boot

Random Access Memory

- RAM is a space of memory on the order of ~10 GiB, which can be used by programs to store main memory.
- Main idea: OS assigns parts of RAM to processes. When a process tries to access a memory location, the OS takes control for a bit and translates to the corresponding spot in RAM.
- Different processes get different parts of RAM, so process A can't access memory of process B even if it tries to (unless there's explicit memory sharing).
- The OS also stores its data in RAM, so it needs to make sure that processes don't access memory it doesn't own.

Virtual Memory: Preview

- Most processes don't use all their address space.
 - They mostly use one block of memory in the code segment, another block in the stack, and maybe a few blocks in the heap/data segments.
- If we gave each process a full 2^{48} bytes of RAM, we won't be able to fit many processes in our RAM.
- Solution: Chop up main memory into "pages" of a few KiB each. When a program tries to access a memory address, have the OS translate the address to its page, and pick up the corresponding data.
 - Ex. A program may be using 1 page for its code, 1 page for its stack, 4 pages for its heap, etc.
- OS as referee: Establish a limit on how many pages a process is allowed to use, and if that limit gets exceeded, don't allow the memory allocation (ex. return NULL on malloc, cause a stack overflow exception).
- If a process tries to access a memory location that hasn't been allocated to them, either create a new page for it, or segfault.

Sharing CPU: Context Switches

- Main idea: In order to handle multiple threads on a single core, we need to have a way to run one thread for a while, then switch to another thread.
 - Goal: A thread should not "notice" that it was paused, so we need to save its state and restore it later.
- Context Switch steps:
 - OS takes control from the old thread
 - OS saves the register values for the old thread
 - OS loads register values for the new thread
 - OS transfers control to the new thread

Sharing CPU: Context Switches

- Context switches tend to be time-consuming, so we generally run them when we already have to wait for other things to happen.
 - Ex. Main memory accesses can take hundreds or thousands of cycles to run, so while waiting for that, we can context switch and run something else for a while
- OS is responsible for deciding when/how often context switches happen, and which threads get access to CPU time.
 - As such, multiple threads run in effectively random order, and potentially a random interlacing of operations.

System Calls

- Functions which can't be directly handled by the program
 - Examples in C: file operations, print statements, `sbrk` (which is used by `malloc`)
 - Examples in RISC-V: `ecalls`
- Handled by transferring control back to the OS in kernel mode for the syscall
- The OS checks if the operation is allowed, and if so, runs the operation for the process.

Today's Agenda

- **Operating Systems and IO**
 - Managing Multiprocessing: Sharing Memory and Compute Time
 - **Interrupts, Exceptions, Trap Handler**
 - Input/Output
 - Boot

Exceptions and Interrupts

Exceptions

- Caused by an event during the execution of the current program.
- **Synchronous**; must be handled immediately.
- Examples:
 - Illegal instruction
 - Divide by zero
 - Write protection violation

"An unexpected event"

Interrupts

- Caused by an event external to the current running program.
- **Asynchronous** to current program; does not need to be handled immediately (but should be soon).
- Examples:
 - Key press
 - Disk I/O

"A request to handle"

Exception or Interrupt? Segmentation Fault

- Recall the segmentation fault
 - A program tries to access a location in memory it should not be allowed to access
- Exception or interrupt?

Exception or Interrupt? Segmentation Fault

- Recall the segmentation fault
 - A program tries to access a location in memory it should not be allowed to access
- Exception or interrupt? **Exception**
 - The **exception handler** takes control immediately and shuts down the program before it can do more harm.
 - Any other program on the system is unaffected: The OS **isolates** bugs to just one program.

Exception or Interrupt? Context Switch

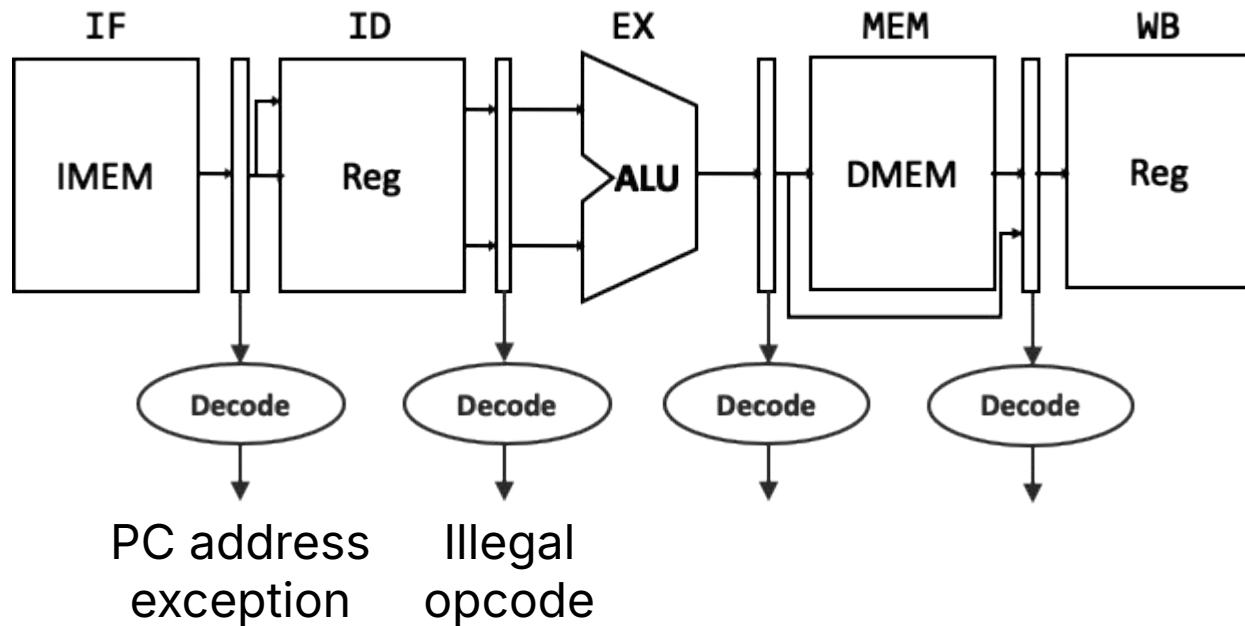
- Recall the context switch
 - In order to handle multiple threads on a single core, we need to have a way to run one thread for a while, then switch to another thread.
 - Goal: A thread should not "notice" that it was paused, so we need to save its state and restore it later.
- 1. Context Switch steps:
- 2. OS takes control from the old thread
- 3. OS saves the register values for the old thread
- 4. OS loads register values for the new thread
- 5. OS transfers control to the new thread
- Exception or interrupt?

Exception or Interrupt? Context Switch

- Recall the context switch
 - In order to handle multiple threads on a single core, we need to have a way to run one thread for a while, then switch to another thread.
 - Goal: A thread should not "notice" that it was paused, so we need to save its state and restore it later.
- Exception or interrupt? **Interrupt**
 - At a high-level:
 - The OS sets a timer (**~1000 cycles**)
 - When it expires, perform a **hardware interrupt**.
 - Trap handler **saves all register values**, including:
 - Program Counter (PC)
 - **Page Table Register** (SPTBR in RV32I)
 - The memory address of the active process's page table.
 - Trap handler then loads in the next process's registers and returns to user mode.

Traps

- A synchronous transfer of control from user to supervisor
- Infer exception cause from faulting instruction and current pipeline stage



Trap Handler

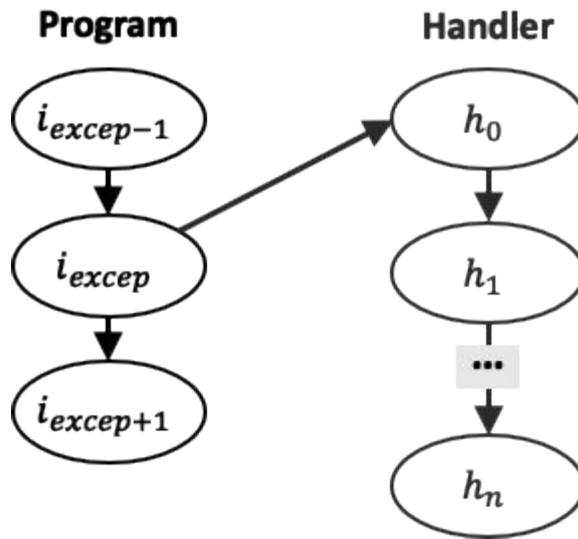
The trap handler is code that services exceptions and interrupts. "Handles" traps.

On the next slide:

1. **Complete** all instructions before the faulting instruction.
2. **Flush** all instructions after the faulting instruction.
 - Like pipeline hazard: convert to noops/"bubbles."
 - Also flush faulting instruction.
3. **Transfer execution to trap handler (runs in supervisor mode).**
 - Optionally return to original program and re-execute instruction.
 - If the trap handler returns, then from the program's point of view it must look like nothing has happened.

Trap Handler

1. Save the state of the current program.
 - Save ALL of the registers!
2. Determine what caused the exception/interrupt.
3. Handle exception/interrupt...

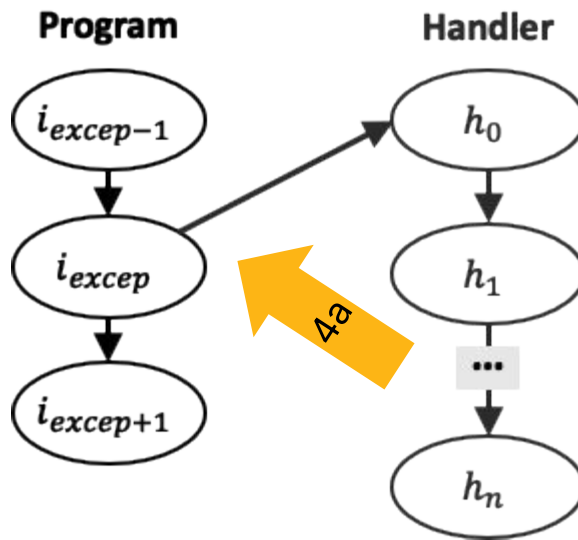


Trap Handler

1. Save the state of the current program.
 - Save ALL of the registers!
2. Determine what caused the exception/interrupt.
3. Handle exception/interrupt...then EITHER:

4a. Continue execution of the program.

- Restore program state.
- Return control to the program.



4b. Terminate the program.

- Free the program resources, etc.
- Schedule a new program.

Today's Agenda

- **Operating Systems and IO**

- Managing Multiprocessing: Sharing Memory and Compute Time
- Interrupts, Exceptions, Trap Handler
- **Input/Output**
- Boot

I/O devices

- I/O constitutes anything that can communicate with your program
 - Inputs:
 - Mouse
 - Keyboard
 - Microphone
 - Camera
 - Outputs:
 - Speakers
 - Display
 - Both:
 - Long-term Memory (Magnetic Drive or SSD)
 - Touch Screens
 - Network/Internet communications
- Communication ranges from a few bytes/second (ex. keyboard) to a few hundred MiB/second (ex. SSD)

I/O devices

- We could define a separate scheme to handle each different I/O device...
- ...or we could **standardize the interface** and let the I/O device handle any complications.
 - **bytestream!!**
- Bytestreams capture many different perspectives:

Program: all I/O gets handled through the file system (ex. fopen, fwrite, fread, fprintf, fclose), though libraries exist to help with interfaces.

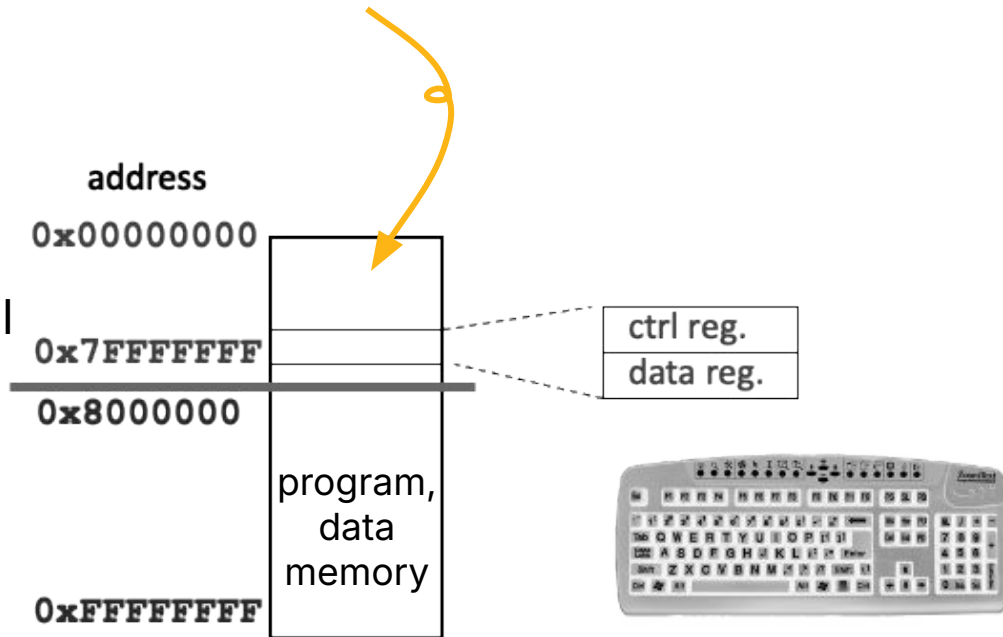
Input: Receive info from the world, and turn it into a bytestream sent to some fixed memory location

Output: Receive a bytestream from some fixed memory location, and translate that into appropriate output

OS: Bring data from/to the I/O device to/from the process every once in a while

I/O devices

- Shared abstraction: I/O devices read/write bytestreams.
- Shared interface is therefore **memory mapped I/O**:
 - This is a portion of address space dedicated to I/O that does **not** contain “regular” memory; rather, it corresponds to registers in I/O devices!
 - Makes it easy to use normal load/store instructions, e.g. lw/sw
 - (Very common, used by RISC-V)
- More on this Wednesday!



Processor: Polls vs Interrupts

Polling

- e.g., “30 times per second”
- Processor reads from Control Register **in loop**
 - Waiting for device to set Ready bit in Control reg (0 → 1)
 - Indicates “data available” or “ready to accept data”
- Processor then loads from (input) or writes to (output) data register
 - I/O device resets control register bit (1 → 0)

Interrupts

- Avoid wasting processor resources for **low data rate** devices (e.g., mouse, keyboard)
- Processor runs as usual
 - Occurs when I/O is ready or needs attention
 - Interrupt current program
 - Transfer control to the trap handler in the operating system

Today's Agenda

- **Operating Systems and IO**
 - Managing Multiprocessing: Sharing Memory and Compute Time
 - Interrupts, Exceptions, Trap Handler
 - Input/Output
 - **Boot**

Intro: Boot

- We talked about how the OS...
 - Functions as an interface between programs, providing communication, protection and synchronization between them
 - Handles multiprocessing / multiprogramming
 - Tackles interrupts and exceptions of various sorts
- But we haven't talked about the first thing that has to happen before these processes...
- **Boot**: the first thing that runs when the computer starts
 - Starts services (more than 100!)
 - File system, network stack, keyboard, etc.

Intro: Boot

- Then, the **BIOS (Basic Input Output System)** firmware:
 - Loads the **bootloader**...
 - ...which loads the **OS kernel**.
1. **BIOS**: Find a storage device and load the first sector (block of data).

```
Diskette Drive B : None          Serial Port(s) : 3F0 2F0
Pri. Master Disk : LBA,ATA 100, 250GB Parallel Port(s) : 370
Pri. Slave Disk : LBA,ATA 100, 250GB DDR at Bank(s) : 0 1 2
Sec. Master Disk : None
Sec. Slave Disk : None

Pri. Master Disk HDD S.M.A.R.T. capability ... Disabled
Pri. Slave Disk HDD S.M.A.R.T. capability ... Disabled

PCI Devices Listing ...
Bus Dev Fun Vendor Device SVID SSID Class Device Class IRQ
-----
0 27 0 8086 2668 1458 A005 0403 Multimedia Device 5
0 29 0 8086 2658 1458 2658 0C03 USB 1.1 Host Cntrlr 9
0 29 1 8086 2659 1458 2659 0C03 USB 1.1 Host Cntrlr 11
0 29 2 8086 265A 1458 265A 0C03 USB 1.1 Host Cntrlr 11
0 29 3 8086 265B 1458 265A 0C03 USB 1.1 Host Cntrlr 5
0 29 7 8086 265C 1458 5006 0C03 USB 1.1 Host Cntrlr 9
0 31 2 8086 2651 1458 2651 0101 IDE Cntrlr 14
0 31 3 8086 266A 1458 266A 0C05 SMBus Cntrlr 11
1 0 0 10DE 0421 10DE 0479 0300 Display Cntrlr 5
2 0 0 1283 8212 0000 0000 0180 Mass Storage Cntrlr 10
2 5 0 11AB 4320 1458 E000 0200 Network Cntrlr 12
ACPI Controllor 9
```


Intro: Boot

- Then, the **BIOS (Basic Input Output System)** firmware:
 - Loads the **bootloader**...
 - ...which loads the **OS kernel**.
1. **BIOS**: Find a storage device and load the first sector (block of data).
 2. **Bootloader** (stored on, e.g., disk): Load the OS kernel from disk into a location in memory and jump into it.
 - 3.
- | | | | |
|------------------|----------------------|------------------|-----------|
| Diskette Drive B | : None | Serial Port(s) | : 3F0 2F0 |
| Pri. Master Disk | : LBA,ATA 100, 250GB | Parallel Port(s) | : 370 |
| Pri. Slave Disk | : LBA,ATA 100, 250GB | DDR at Bank(s) | : 0 1 2 |
| Sec. Master Disk | : None | | |
| Sec. Slave Disk | : None | | |

```

Diskette Drive B : None
Pri. Master Disk : LBA,ATA 100, 250GB Parallel Port(s) : 3F0 2F0
Pri. Slave Disk : LBA,ATA 100, 250GB DDR at Bank(s) : 0 1 2
Sec. Master Disk : None
Sec. Slave Disk : None

Pri. Master Disk HDD S.M.A.R.T. capability ... Disabled
Pri. Slave Disk HDD S.M.A.R.T. capability ... Disabled

PCI Devices Listing ...
Bus Dev Fun Vendor Device SUID SSID Class Device Class IRQ
0 27 0 8086 2668 1458 A005 0403 Multimedia Device 5
0 29 0 8086 2658 1458 2658 0C03 USB 1.1 Host Cntrlr 9
0 29 1 8086 2659 1458 2659 0C03 USB 1.1 Host Cntrlr 11
0 29 2 8086 265A 1458 265A 0C03 USB 1.1 Host Cntrlr 11
0 29 3 8086 265B 1458 265A 0C03 USB 1.1 Host Cntrlr 5
0 29 7 8086 265C 1458 5006 0C03 USB 1.1 Host Cntrlr 9
0 31 2 8086 2651 1458 2651 0101 IDE Cntrlr 14
0 31 3 8086 266A 1458 266A 0C05 SMBus Cntrlr 11
1 0 0 10DE 0421 10DE 0479 0300 Display Cntrlr 5
2 0 0 1283 8212 0000 0000 0180 Mass Storage Cntrlr 10
2 5 0 11AB 4320 1458 E000 0200 Network Cntrlr 12
ACPI Controller 9

```

Intro: Boot

- Then, the **BIOS (Basic Input Output System)** firmware:
 - Loads the **bootloader**...
 - ...which loads the **OS kernel**.
1. **BIOS**: Find a storage device and load the first sector (block of data).
 2. **Bootloader** (stored on, e.g., disk): Load the OS kernel from disk into a location in memory and jump into it.
 3. **OS Boot**: Initialize services, drivers, etc.
 - 4.
- ```
Diskette Drive B : None Serial Port(s) : 3F0 2F0
Pri. Master Disk : LBA,ATA 100, 250GB Parallel Port(s) : 370
Pri. Slave Disk : LBA,ATA 100, 250GB DDR at Bank(s) : 0 1 2
Sec. Master Disk : None
Sec. Slave Disk : None

Pri. Master Disk HDD S.M.A.R.T. capability ... Disabled
Pri. Slave Disk HDD S.M.A.R.T. capability ... Disabled
```

```

Diskette Drive B : None
Pri. Master Disk : LBA,ATA 100, 250GB Parallel Port(s) : 3F0 2F0
Pri. Slave Disk : LBA,ATA 100, 250GB DDR at Bank(s) : 0 1 2
Sec. Master Disk : None
Sec. Slave Disk : None

Pri. Master Disk HDD S.M.A.R.T. capability ... Disabled
Pri. Slave Disk HDD S.M.A.R.T. capability ... Disabled

PCI Devices Listing ...
Bus Dev Fun Vendor Device SUID SSID Class Device Class IRQ
0 27 0 8086 2668 1458 A005 0403 Multimedia Device 5
0 29 0 8086 2658 1458 2658 0C03 USB 1.1 Host Cntrlr 9
0 29 1 8086 2659 1458 2659 0C03 USB 1.1 Host Cntrlr 11
0 29 2 8086 265A 1458 265A 0C03 USB 1.1 Host Cntrlr 11
0 29 3 8086 265B 1458 265A 0C03 USB 1.1 Host Cntrlr 5
0 29 7 8086 265C 1458 5006 0C03 USB 1.1 Host Cntrlr 9
0 31 2 8086 2651 1458 2651 0101 IDE Cntrlr 14
0 31 3 8086 266A 1458 266A 0C05 SMBus Cntrlr 11
1 0 0 10DE 0421 10DE 0479 0300 Display Cntrlr 5
2 0 0 1283 8212 0000 0000 0180 Mass Storage Cntrlr 10
2 5 0 11AB 4320 1458 E000 0200 Network Cntrlr 12
ACPI Controller 9

```

# Intro: Boot

- Then, the **BIOS (Basic Input Output System)** firmware:
  - Loads the **bootloader**...
  - ...which loads the **OS kernel**.
- 1. **BIOS**: Find a storage device and load the first sector (block of data).
- 2. **Bootloader** (stored on, e.g., disk): Load the OS kernel from disk into a location in memory and jump into it.
- 3. **OS Boot**: Initialize services, drivers, etc.
- 4. **Init**: Launch an application (e.g., Terminal/Desktop/...) that waits for input in loop.

```
Diskette Drive B : None Serial Port(s) : 3F0 2F0
Pri. Master Disk : LBA,ATA 100, 250GB Parallel Port(s) : 370
Pri. Slave Disk : LBA,ATA 100, 250GB DDR at Bank(s) : 0 1 2
Sec. Master Disk : None
Sec. Slave Disk : None
```

```
Pri. Master Disk HDD S.M.A.R.T. capability ... Disabled
Pri. Slave Disk HDD S.M.A.R.T. capability ... Disabled
```

#### PCI Devices Listing ...

| Bus | Dev | Fun | Vendor | Device | SUID | SSID | Class | Device Class        | IRQ |
|-----|-----|-----|--------|--------|------|------|-------|---------------------|-----|
| 0   | 27  | 0   | 8086   | 2668   | 1458 | A005 | 0403  | Multimedia Device   | 5   |
| 0   | 29  | 0   | 8086   | 2658   | 1458 | 2658 | 0C03  | USB 1.1 Host Cntrlr | 9   |
| 0   | 29  | 1   | 8086   | 2659   | 1458 | 2659 | 0C03  | USB 1.1 Host Cntrlr | 11  |
| 0   | 29  | 2   | 8086   | 265A   | 1458 | 265A | 0C03  | USB 1.1 Host Cntrlr | 11  |
| 0   | 29  | 3   | 8086   | 265B   | 1458 | 265A | 0C03  | USB 1.1 Host Cntrlr | 5   |
| 0   | 29  | 7   | 8086   | 265C   | 1458 | 5006 | 0C03  | USB 1.1 Host Cntrlr | 9   |
| 0   | 31  | 2   | 8086   | 2651   | 1458 | 2651 | 0101  | IDE Cntrlr          | 14  |
| 0   | 31  | 3   | 8086   | 266A   | 1458 | 266A | 0C05  | SMBus Cntrlr        | 11  |
| 1   | 0   | 0   | 10DE   | 0421   | 10DE | 0479 | 0300  | Display Cntrlr      | 5   |
| 2   | 0   | 0   | 1283   | 8212   | 0000 | 0000 | 0180  | Mass Storage Cntrlr | 10  |
| 2   | 5   | 0   | 11AB   | 4320   | 1458 | E000 | 0200  | Network Cntrlr      | 12  |
|     |     |     |        |        |      |      |       | ACPI Controller     | 9   |

# CS 162 preview

- CS 162 is our Operating Systems class
- Goes into much greater detail about each of these components
- Projects focus on developing an OS of your own in C (Pintos).
  - Very big starter code, very difficult to debug

# Conclusion

- The OS allows each program to work independently, as if it was the only thing on the system.
  - No program should interact with any other (unless intentional)
  - No program should be allowed to do illegal things that break your computer (even if the program tries)
  - Programs don't need to worry about interfacing with a bunch of different I/O devices in different ways
  - Programs don't need to handle their own loading, setting up memory, etc.
- Why? **Abstraction.**
  - Different people make different programs run on your device. No need for those people to cooperate/manage interactions between their programs.
  - Letting the OS handle some complexity lets you as the programmer focus more on logic of your program; reduced overhead to innovation.
- Costs a bit of memory/clock time, but almost always worth it.